

c-code-conventions

C Code Conventions and Guards

This document is a portable extraction of the C, Win32, style, analysis, and guard policies from the source repository. It is intended to be copied into other Windows C projects and adapted by replacing project-specific names, paths, and prefixes.

The defaults assume a Windows-first C codebase using MSBuild, MSVC, Clang tooling, Win32 APIs, and PowerShell guard scripts.

1. Enforcement Model

Prefer rules that can be checked by scripts over rules that only live in code review. When a warning, guard, or analysis check reports a real issue, fix the underlying code shape instead of suppressing the diagnostic.

Do not weaken checks to get a build through:

- Do not suppress actionable warnings locally with pragmas, `/wd...`, or project-local warning disable lists.
- Do not turn off warnings-as-errors for one project or configuration.
- Do not disable LTCG/WPO, analysis, tidy, formatting, hooks, or line-ending checks as a workaround.
- Do not bypass hooks with `--no-verify` unless the user explicitly authorized that operation for the immediate task.
- Do not use destructive git operations such as `reset --hard`, `clean -fdx`, branch deletion, or force push without explicit authorization.

If a rule is wrong, correct it with evidence and update the incident/history ledger. Do not silently remove the guard.

2. Repository Text Policy

All repository text files must use CRLF line endings. This avoids Visual Studio and Windows tooling churn in Win32/MSBuild/WiX repositories.

Encode this policy in:

- `.editorconfig: end_of_line = crlf, insert_final_newline = true.`

- `.gitattributes`: `* text=auto eol=crlf`, with binary extensions marked as `binary`.
- A line-ending guard that can check or fix project-referenced files.
- Hook wrappers that reject writes containing bare LF line endings.

Suggested commands:

```
. \tools\line-ending-guard.ps1 -Check -ProjectReferences
. \tools\line-ending-guard.ps1 -Fix -ProjectReferences
```

The guard should be able to:

- scan files referenced by `.sln`, `.vcxproj`, and `.wixproj`;
- ignore known binary extensions and generated/build directories;
- detect bare LF bytes without re-encoding text;
- normalize only line endings when fixing;
- run as a hook for edit/write tools and block new bare LF content.

3. Build and Analysis Baseline

Keep common build policy centralized in shared MSBuild props. Do not duplicate runtime, warning, analysis, or warning-disable policy in individual projects.

Recommended defaults:

- Use the static Visual C++ runtime consistently: `/MTd` for Debug and `/MT` for Release-style builds.
- Treat warnings as errors for project code.
- Enable MSVC native code analysis and CppCoreCheck through shared props.
- Enable `NativeRecommendedRules.ruleset`.
- Enable arithmetic analysis such as `CppCoreCheckArithmeticRules.ruleset`.
- Keep external SDK/header analysis disabled so platform header noise does not mask source diagnostics.
- Keep C26451-style arithmetic overflow warnings enabled and fix them by widening operands before the arithmetic happens.

Warnings such as C26494/C26495 can be disabled centrally when they are proven to be noise for a deliberate C89 declaration style. Do not use that as a general excuse to suppress real source warnings.

4. ReleasePerf, C5045, and No-CRT Code

Spectre warning C5045 must be fixed structurally. Do not suppress C5045 with `<DisableSpecificWarnings>`, `#pragma warning(disable:5045)`, `/wd5045`, or by turning off warnings-as-errors.

Fix C5045 one reported site at a time:

1. Build.
2. Inspect the first warning C5045 and its note: lines.
3. Repair the local dataflow.
4. Rebuild and repeat.

C5045-sensitive code is any code where a bounds, range, or size check proves a value safe and that value later feeds a call, array index, pointer arithmetic, return value, or structure field access. Convert guard-proved values into safe stack locals before a call-bearing path consumes them.

In no-CRT or hook/sidecar code, avoid helper dependencies that the target cannot link:

- Avoid direct 64-bit division/modulo that emits helper symbols such as `__alldiv` or `__aulldiv`.
- Avoid CRT math calls such as `sin` and `cos`.
- Prefer existing integer/manual helpers or Win32/GDI APIs.

C4711, "selected for automatic inline expansion," is optimizer noise in LTCG builds. If it must be disabled, do so only as a central shared build-policy exception, not with source pragmas or project-local suppressions.

5. Formatting Baseline

Use clang-format for mechanical C-family layout, then run a project-specific post-pass for the stricter table alignment rules that clang-format cannot express.

Recommended `.clang-format` baseline:

```
---
Language: Cpp
BasedOnStyle: Microsoft
ColumnLimit: 120
UseTab: Never
TabWidth: 4
IndentWidth: 4
ContinuationIndentWidth: 4
```

BreakBeforeBraces: Allman
IndentCaseLabels: **false**
IndentPPDirectives: None

SortIncludes: Never
IncludeBlocks: Preserve

DerivePointerAlignment: **false**
PointerAlignment: Left

AlignAfterOpenBracket: Align
AlignConsecutiveAssignments:
 Enabled: **true**
 AcrossEmptyLines: **false**
 AcrossComments: **false**
 AlignCompound: **true**
 PadOperators: **true**

AlignConsecutiveMacros:
 Enabled: **true**
 AcrossEmptyLines: **false**
 AcrossComments: **false**

AlignConsecutiveDeclarations:
 Enabled: **true**
 AcrossEmptyLines: **true**
 AcrossComments: **true**
 AlignFunctionPointers: **true**

BinPackArguments: **false**
BinPackParameters: **false**
AllowAllParametersOfDeclarationOnNextLine: **false**
Cpp11BracedListStyle: **false**

AllowShortBlocksOnASingleLine: Never
AllowShortCaseLabelsOnASingleLine: **false**
AllowShortFunctionsOnASingleLine: Empty
AllowShortIfStatementsOnASingleLine: WithoutElse
AllowShortLoopsOnASingleLine: **false**

SpaceBeforeParens: ControlStatements
SpaceBeforeAssignmentOperators: **true**
SpaceAfterCStyleCast: **false**
SpacesInParentheses: **false**
SpacesInSquareBrackets: **false**

BreakStringLiterals: **false**

```
ReflowComments: false
MaxEmptyLinesToKeep: 1
InsertNewlineAtEOF: true
```

The formatter post-pass should handle:

- declaration identifier alignment;
- structure member alignment;
- selected assignment/operator alignment;
- repeated guard-return alignment;
- message-dispatch table alignment;
- `EXTERN_C <type> CFORCEINLINE` declaration alignment when used.

Run formatting with:

```
.\tools\format.ps1
.\tools\format.ps1 -Check
```

6. clang-tidy Baseline

At minimum, enable one-declarator-per-statement checks:

```
---
Checks: '-*,readability-isolate-declaration,bugprone-assignment-in-if-
condition'
WarningsAsErrors: 'readability-isolate-declaration,bugprone-assignment-in-if-
condition'
HeaderFilterRegex: '.*'
FormatStyle: file
...
```

Strict mode should run on translation units, not standalone headers. Header-only runs often produce noisy diagnostics for macros and constants consumed by `.c` or `.rc` files.

For broad C audits, prefer a focused list such as:

```
bugprone-*, clang-analyzer-*, clang-diagnostic-*, cert-*, misc-*,
readability-isolate-declaration
```

Disable known noisy or non-C policy families unless the project has explicitly chosen them: broad readability, magic-number, identifier-length, C++ guideline,

vendor-family, and modernize checks are usually poor defaults for Win32 C.

7. Source Layout

In `.c` files, includes, defines, source-level type definitions, file-scope prototypes, and global/static data declarations belong in the preamble before the first function definition.

Do not introduce these halfway through a translation unit:

- `#include`
- `#define`
- `typedef`
- `struct`
- `enum`
- `static struct`
- file-scope prototypes
- global/static data declarations

8. Defines

Adjacent object-like `#define` rows with replacement values form a table. Align the replacement-value column across the contiguous block. Header guards and feature toggles without replacement values are not part of this rule.

```
/* correct */
#define APP_COLOR_SELECTION      RGB(31, 86, 136)
#define APP_COLOR_FLAME_SELECTION RGB(44, 74, 104)
#define APP_COLOR_SOURCE_HIT_TEXT RGB(255, 230, 210)

/* wrong */
#define APP_COLOR_SELECTION RGB(31, 86, 136)
#define APP_COLOR_FLAME_SELECTION RGB(44, 74, 104)
#define APP_COLOR_SOURCE_HIT_TEXT RGB(255, 230, 210)
```

9. Variable Declarations

All stack variables must be declared at the top of their block scope, C89 style, before executable statements. This applies to every block, including nested blocks inside `if`, `for`, `while`, and `switch` cases.

`UNREFERENCED_PARAMETER(...)` markers belong immediately after declarations and before executable statements.

```

/* correct */
void Foo(void)
{
    int nIndex;
    int nCount;

    nCount = Bar();
    for (nIndex = 0; nIndex < nCount; nIndex++) { ... }
}

/* wrong */
void Foo(void)
{
    int nCount = Bar();
    int nIndex;
    ...
}

/* wrong */
for (int nIndex = 0; nIndex < nCount; nIndex++) { ... }

```

Within a declaration preamble:

- declare locals in the order their identifiers are first used;
- align consecutive declaration identifier columns;
- use one declarator per statement;
- do not initialize stack locals in declarations.

Assign values in the executable stream after the complete declaration preamble.

```

/* correct */
LONG   lr;
HKEY   hKey;
DWORD  cbData;
DWORD  dwType;

hKey = NULL;
lr   = RegOpenKeyEx(HKEY_CURRENT_USER, APP_REG_KEY, 0, KEY_READ, &hKey);

/* wrong */
LONG   lr;
HKEY   hKey = NULL;
DWORD  cbData;
DWORD  dwType;

```

Naked scopes are banned. Do not add anonymous `{ ... }` blocks only to contain temporary declarations. Hoist the declarations to the owning block.

10. Names

Prefer names that state the domain quantity being manipulated.

UI geometry, layout, and drawing code must not use bare `w`, `h`, `r`, `g`, or `gs` when the value is width, height, radius, glyph text, or glyph size. Use names such as `nWidth`, `nHeight`, `nRadius`, `szGlyph`, and `nGlyphSize`.

Do not encode design constants as magic numbers in the assignment stream. Use an existing named constant or add one.

For Win32 structure pairs where one type is the extended `...EX / ...Ex` form of a base type, reserve the base abbreviation for the base structure and add a trailing `x` for the extended structure. For example, use `wc` for `WNDCLASS` and `wcx` for `WNDCLASSEX`.

Use `ARRAYSIZE(...)` for compile-time array element counts. Do not use `_countof(...)`.

11. Pointer Values

When the value pointed to by a pointer participates in an expression, write the dereference as `(*p)`. If a pointer advance and a pointed-to value read or write both happen, split them into separate statements.

```
/* correct */
fHasSpace = (((*pcchUri) + 1) < cchUri);
szOut[(*pcchOut)] = ch;
(*pcchOut) += 1;

(*pWrite) = (*pRead);
pWrite++;
pRead++;

/* wrong */
fHasSpace = *pcchUri + 1 < cchUri;
szOut[*pcchOut] = ch;
*pWrite++ = *pRead++;
```

For `sizeof` over a pointer dereference, write `sizeof((*pThing))`, not `sizeof(*pThing)`.

12. Function Calls and Table Alignment

Do not put ternary expressions in function-like call arguments. Compute the selected value in a named local before the call.

```
cbPayload = 0;
if (1 < cb) cbPayload = (DWORD)(cb - 1);
fWrote = WriteFile(hFile, pszUtf8, cbPayload, &cbWritten, NULL);
```

When adjacent statements form a table of repeated operations, align the semantically matching columns inside that local cluster.

```
AppWrite(hFile, APP_FILE_MAGIC, APP_FILE_MAGIC_LEN);
AppWrite(hFile, &dwVersion, sizeof(dwVersion));
AppWrite(hFile, &dwTicksPerUnit, sizeof(dwTicksPerUnit));
AppWrite(hFile, &dwPid, sizeof(dwPid));
```

Adjacent one-line guard returns are part of the same visual-table rule:

```
if (AppNameMatches(szPath, APP_HOST_EXE)) return TRUE;
if (AppNameMatches(szPath, APP_SIDEAR_EXE)) return TRUE;
if (!AppTryEnterScope()) return FALSE;
```

Use the generic Win32/TCHAR entrypoint when the surrounding data is `TCHAR`, `LPTSTR`, `LPCTSTR`, or `TEXT(...)`. Do not call explicit `A` or `W` variants unless the API has no generic macro, the function signature explicitly uses narrow or wide types, or the code is deliberately pinned to a narrow/wide external format.

13. Conditions

Use Yoda conditions for comparisons against constants, literals, and macro values. Put the constant on the left. For ordered comparisons, flip the operator when moving the constant.

```
if (0 == result) { ... }
if (MAX_PATH > cchPath) { ... }
```

Do not compare `BOOL` question values to `TRUE` or `FALSE`. Use the question directly for the true case and `!` for the false case.

Bitmask presence tests are not equality comparisons:

- Put the mask constant or macro on the left.
- Do not compare the masked value to zero.
- In control-flow conditions, assign the test to a named stack-local `BOOL` question first.
- When assigning to `BOOL`, canonicalize with `!!`.

```
/* correct */
fShiftDown = !(MK_SHIFT & uKeyFlags);
if (fShiftDown) { ... }

/* wrong */
if (0 != (uKeyFlags & MK_SHIFT)) { ... }
if (MK_SHIFT & uKeyFlags) { ... }
fShiftDown = (BOOL)(MK_SHIFT & uKeyFlags);
fShiftDown = (MK_SHIFT & uKeyFlags) ? TRUE : FALSE;
```

Operators that already produce boolean results do not need `!!` for `BOOL` storage. Logical operators and comparison operators produce `0` or `1`.

In consecutive comparison assignments or condition rows, align the comparison operator inside the question:

```
fDownKey    = (VK_DOWN    == vk);
fUpKey      = (VK_UP      == vk);
fReturnKey  = (VK_RETURN  == vk);
fEscapeKey  = (VK_ESCAPE  == vk);
```

In consecutive simple arithmetic assignments, align the first binary arithmetic operator when the operator is the same across the group:

```
nWidth  = rc.right - rc.left;
nHeight = rc.bottom - rc.top;
```

When a compound boolean expression contains comparisons, parenthesize each comparison term. Also parenthesize negated terms inside any compound condition.

```
fBoundsValid = (0 < nWidth) && (0 < nHeight);
fInvalidArgs = (!fHasJson) || (!fHasEntry) || (!fHasOutput);
```

Complex conditionals must be split into named stack-local `BOOL` questions. Each value should answer one specific question.

```

BOOL fHasWindow;
BOOL fWindowVisible;

fHasWindow      = (NULL != hwnd);
fWindowVisible = fHasWindow && IsWindowVisible(hwnd);
if (fWindowVisible) { ... }

```

14. Switches

Order `case` labels from least to greatest by resolved case value. This also applies to Win32 message-cracker dispatch rows inside `switch` blocks. Message tables are dispatch tables, not alphabetic lists.

15. Win32 Message Crackers

Win32 window procedures and subclass procedures must use message-cracker dispatch. Do not build message dispatch as a run of `if (WM_* == uMsg)` checks. Use `switch (uMsg)` with `HANDLE_MSG`, project-specific cracker macros, or a local cracker macro when `windowsx.h` does not provide one.

Message handler routines use the window-class prefix followed by `_On*`, such as `MainWindow_OnPaint`, `MainWindow_OnSize`, and `MainWindow_OnNcCreate`. Shared helpers that are not direct message handlers do not use the `_On*` form.

Each distinct Win32 message in a dispatch table must have its own cracker row and its own direct `_On*` handler when the messages have distinct semantics, even if implementations are similar. For example, `WM_MOUSEWHEEL` and `WM_MOUSEHWHEEL` should dispatch to separate direct handlers.

Direct `_On*` handlers must stay thin when multiple messages share behavior. Move shared behavior to a helper and have each direct handler pass only the message-specific facts decoded by its cracker.

When replacing a raw message case that did work and then allowed `DefWindowProc` to run, preserve default processing explicitly in the direct handler with the matching `FORWARD_WM_*` macro. Do not depend on fallthrough and do not manually rebuild `wParam / lParam` for `DefWindowProc`.

Dispatch tables must align the handler/function argument column:

```

switch (uMsg)
{
    HANDLE_MSG(hwnd, WM_CREATE,                MainWindow_OnCreate);

```

```

    HANDLE_MSG(hWnd, WM_DESTROY,      MainWindow_OnDestroy);
    HANDLE_MSG(hWnd, WM_SIZE,          MainWindow_OnSize);
    HANDLE_MSG(hWnd, WM_PAINT,          MainWindow_OnPaint);
    HANDLE_MSG(hWnd, WM_SYSCOLORCHANGE, MainWindow_OnSysColorChange);
    HANDLE_MSG(hWnd, WM_SETTINGCHANGE, MainWindow_OnSettingChange);
    HANDLE_MSG(hWnd, WM_MOUSEWHEEL,     MainWindow_OnMouseWheel);
    HANDLE_MSG(hWnd, WM_MOUSEHWHEEL,    MainWindow_OnMouseHWheel);
    HANDLE_MSG(hWnd, WM_THEMECHANGED,   MainWindow_OnThemeChanged);
    HANDLE_MSG(hWnd, WM_DWMCOLORIZATIONCOLORCHANGED,
    MainWindow_OnDwmColorizationColorChanged);
}

```

If `windowsx.h` does not provide a needed cracker, add the missing trivial `HANDLE_WM_*` and `FORWARD_WM_*` macros to a shared message header. Custom crackers must include a `windowsx.h`-style class callback comment immediately before the macro definition:

```

#ifndef HANDLE_WM_THEMECHANGED
/* void Cls_OnThemeChanged(HWND hwnd) */
#define HANDLE_WM_THEMECHANGED(hWnd, wParam, lParam, fn) \
    ((void)(wParam), (void)(lParam), (fn)(hWnd), 0L)
#endif

#ifndef FORWARD_WM_THEMECHANGED
/* void Cls_OnThemeChanged(HWND hwnd) */
#define FORWARD_WM_THEMECHANGED(hWnd, fn) \
    ((void)(fn)((hWnd), WM_THEMECHANGED, 0, 0))
#endif

```

16. C5045-Resistant Dataflow

Counted loops whose bodies contain any function-like call or macro call must be post-tested. Do not use a pre-tested relational `while` or `for` loop for a variable-count call-bearing loop.

```

/* correct */
BOOL fHasRows;

fHasRows = (0u != cRows);
if (fHasRows)

```

```

{
    do
    {
        DrawRow(iRow);
        iRow++;
    } while (cRows > iRow);
}

/* wrong */
while (cRows > iRow)
{
    DrawRow(iRow);
    iRow++;
}

```

Prefer branchless safe-value construction for sensitive dataflow. Build a safe index, count, denominator, pointer, empty rectangle, scratch object, or neutral scalar before later calls consume it.

```

BOOL fNodeValid;
DWORD dwNodeMask;
DWORD iNodeSafe;

fNodeValid = (iNode < cNodes);
dwNodeMask = 0u - (DWORD)fNodeValid;
iNodeSafe = iNode & dwNodeMask;
pNode = &rgNodes[iNodeSafe];
DrawNode(pNode);

```

Do not use an early return as the only safety proof immediately before call-heavy code. Null pointer precondition returns remain acceptable. For range, size, count, and index checks, create safe locals and let invalid paths use neutral values.

Any non-constant array index derived from runtime state must either be bounded by the local post-tested loop count or have a named safe-index local before the first index operation.

Capacity-limited append/reserve helpers must not branch directly before returning a pointer or index. Use a stack scratch object plus a mask-selected real/scratch pointer, then update counters branchlessly.

```

fFull = (ROW_STORE_COUNT <= count);
uFullMask = (ULONG_PTR)0u - (ULONG_PTR)fFull;

```

```

pReal      = &rows[count & (ROW_STORE_COUNT - 1u)];
pRow       = (ROW*)((ULONG_PTR)&scratch & uFullMask) | ((ULONG_PTR)pReal &
~uFullMask));
SecureZeroMemory(pRow, sizeof(*pRow));
count += (DWORD)!fFull;
return (ROW*)((ULONG_PTR)pRow & ~uFullMask);

```

When an API returns a usable buffer length, do not recompute and range-check a second length that then feeds formatting or pointer arithmetic. Mask the API-returned length into a safe local, use that local in the call, then validate after the call if needed.

17. Win32 UI and Rendering

Platform target is a first-class axis. A platform target defines where the UI runs and which host services are available. Keep it separate from renderer, layout engine, control catalog, and visual style.

For Win32 targets:

- UI Automation backend is UIA.
- The platform owns assumptions about message loop, threading, DPI, text scale, theme source, high contrast, reduced motion, pointer/touch/pen availability, IME, packaging, and security boundaries.
- Renderer and layout choices must be filtered by what the Win32 host actually supports.

Custom Win32 UI should use retained ownership for layout and state. Derive geometry from canonical state during `WM_SIZE` and DPI/theme changes. Avoid incremental resize drift.

Rendering rules:

- Paint the background and foreground in one coherent path per surface.
- Use dirty rectangles deliberately.
- Do not hide repaint defects with `WM_SETREDRAW`, child-window moves, `RDW_ALLCHILDREN`, `WS_EX_COMPOSITED`, `SWP_NOCOPYBITS`, or broad redraw suppression.
- Text must remain legible before, during, and after interaction.
- If a custom retained UI surface needs high-refresh animation, composition, or precise present timing, consider a DXGI flip-model renderer. GDI is a fallback renderer, not the long-term high-refresh path.

Owner-drawn UI modules should not use visible standard Win32 child controls as their visual implementation. Top-level host windows and hidden owner windows are acceptable boundaries; visible content inside the owned surface should be drawn by the owning paint path. If this policy applies, guard against:

- `WS_CHILD` in owner-drawn UI modules;
- common-control classes such as `WC_LISTVIEW`, `WC_TREEVIEW`, `WC_TABCONTROL`, `WC_HEADER`, `WC_BUTTON`, and `WC_EDIT`;
- common-control helper macros such as `ListView_*`, `TreeView_*`, `TabCtrl_*`, `Header_*`, `Button_*`, `Edit_*`, and `ComboBox_*`;
- `WM_SETREDRAW`;
- `BeginDeferWindowPos`, `DeferWindowPos`, and `EndDeferWindowPos`;
- `RDW_ALLCHILDREN`, `WS_EX_COMPOSITED`, and `SWP_NOCOPYBITS`.

18. Theme and Accessibility

All user-facing Win32 UI must understand the user's current Windows app theme and adapt dynamically. The default theme mode is `System`, not a hard-coded light or dark palette.

Required theme behavior:

- Read the current Windows app theme and derive effective light/dark resources.
- Observe `WM_SETTINGCHANGE`, `WM_THEMECHANGED`, DWM colorization changes, high contrast changes, and DPI changes as resource invalidation triggers.
- Recompute theme resources without recreating logical UI state, selection, focus, scroll position, or command state.
- Keep manual `Light` and `Dark` overrides distinct from `System`.
- Apply the effective theme to custom content, popup hosts, standard child controls, supported non-client chrome, focus visuals, selection, disabled state, text, borders, and accessibility contrast.

All user-facing UI must be keyboard reachable, screen-reader reachable, and testable through Microsoft UI Automation. Every interactive control, owner-drawn region, popup, dialog, list item, command, status surface, and custom renderer-hosted element should specify:

- accessible name;
- control type;
- automation id;
- bounding rectangle;

- enabled and focus state;
- keyboard focusability;
- help text where useful;
- required UIA patterns such as Invoke, Toggle, Selection, Value, RangeValue, Text, Grid, Table, Scroll, and ExpandCollapse;
- UIA events for focus, invoke, selection, value changes, property changes, structure changes, and live-region updates.

19. Guard Checklist

These guard categories are reusable across C projects:

- Line endings: reject bare LF in text files and provide project-reference check/fix modes.
- Circumvention: reject warning suppression, check disabling, destructive git shortcuts, hook bypasses, and guard tampering.
- Convention guard: scan source for high-signal C convention violations.
- Formatting guard: run clang-format plus project-specific post-format passes and fail in `-Check` mode when output would differ.
- clang-tidy guard: enforce one declarator per statement and targeted bugprone checks; run strict checks on translation units only.
- MSVC analysis guard: verify shared props/ruleset still enable native analysis, CppCoreCheck, arithmetic warnings, external-header suppression, and project warning policy.
- Owner-draw guard: when a module is designated owner-drawn, reject standard child-control surfaces and redraw-suppression workarounds.

High-signal convention guard patterns:

- `sizeof(\* -> use sizeof((*p))` .
- generic locals such as `int i;` -> use domain names.
- `FALSE !=` , `TRUE ==` , `FALSE ==` , `TRUE !=` -> do not compare `BOOL` questions to `TRUE` or `FALSE` .
- raw bitmask tests in `if / while` -> assign to a named `BOOL` question first.
- bitmask comparisons to zero -> treat presence as a question, not equality.
- `(BOOL)(MASK & flags)` -> use `!!(MASK & flags)` .
- `? TRUE : FALSE` and `? FALSE : TRUE` -> remove redundant boolean ternaries.
- `DefWindowProc(..., WM_...)` in handlers -> use matching `FORWARD_WM_*` .
- missing callback comments above custom `HANDLE_WM_*` / `FORWARD_WM_*` crackers.

- project-local C5045 suppression in props, source pragmas, or compiler flags.

Complex-condition reporting can be advisory rather than hard-fail. A useful pattern is to report `if`, `while`, or `return` rows that combine `&&` / `||` with comparisons, `sizeof`, or negation, then review whether named stack-local questions would make the code clearer and safer.

20. C5045 Rationale

C5045 exists because Spectre v1 bounds-check bypass is a dataflow problem, not just an array-indexing problem. A branch that proves a value safe can be mispredicted, and the supposedly safe value can still feed a speculative load, call, pointer computation, or return path.

The practical goal is structural mitigation: shape the C code so later operations consume safe values even on the wrong speculative path. Branchless mask construction is the core idiom:

```
fValid = (iValue < cValues);  
dwMask = 0u - (DWORD)fValid;  
iSafe = iValue & dwMask;
```

This turns a branch-dependent safety claim into a value-dependent safety fact. The branch may still exist for control flow, but the branch is no longer the only thing protecting the later access.

Some patterns are useful style but are not Spectre mitigations by themselves:

- Yoda conditions prevent accidental assignment; they do not stop speculation.
- `CFORCEINLINE` is an optimization and ABI/control-flow decision; it is not a Spectre policy.
- `SecureZeroMemory` is correct for erasing sensitive buffers, but it does not fix bounds-check bypass.
- An early return is fine for simple null preconditions, but a range early return is not enough when the checked value later feeds calls or loads.

This policy does not claim cryptographic constant-time behavior. The narrower requirement is that optimized warning-clean builds avoid C5045 dataflow shapes and new C code should not keep reintroducing them.